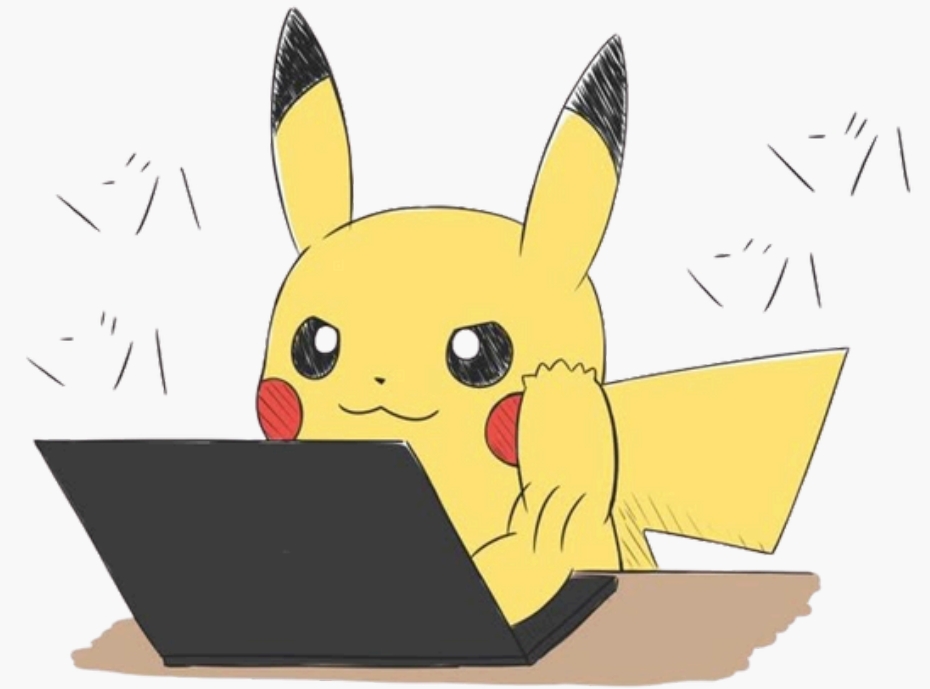


Projeto Final - Jogo Pokemon



Disciplina: Programação e Desenvolvimento de Software II

Professor: Wagner Cipriano da Silva

Grupo 15:

- Arthur Henrique Liberato Lima
- Allan Victor Matos Temponi
- Igor Maia Mendes
- Paulo Henrique Cardoso da Silva

Introdução

O Projeto consiste em um jogo Pokemon no estilo dress to impress. Os avatares possuem atributos bases e que podem ser melhorados utilizando itens que são disponibilizadas ao usuário. São colocados frente a frente oponentes selecionados aleatoriamente para batalhar.

Objetivos:

- Organizar dados complexos;
- Garantir persistência (salvar e carregar jogo);
- Usar boas praticas de programação modular e orientação a objetos;

Características:

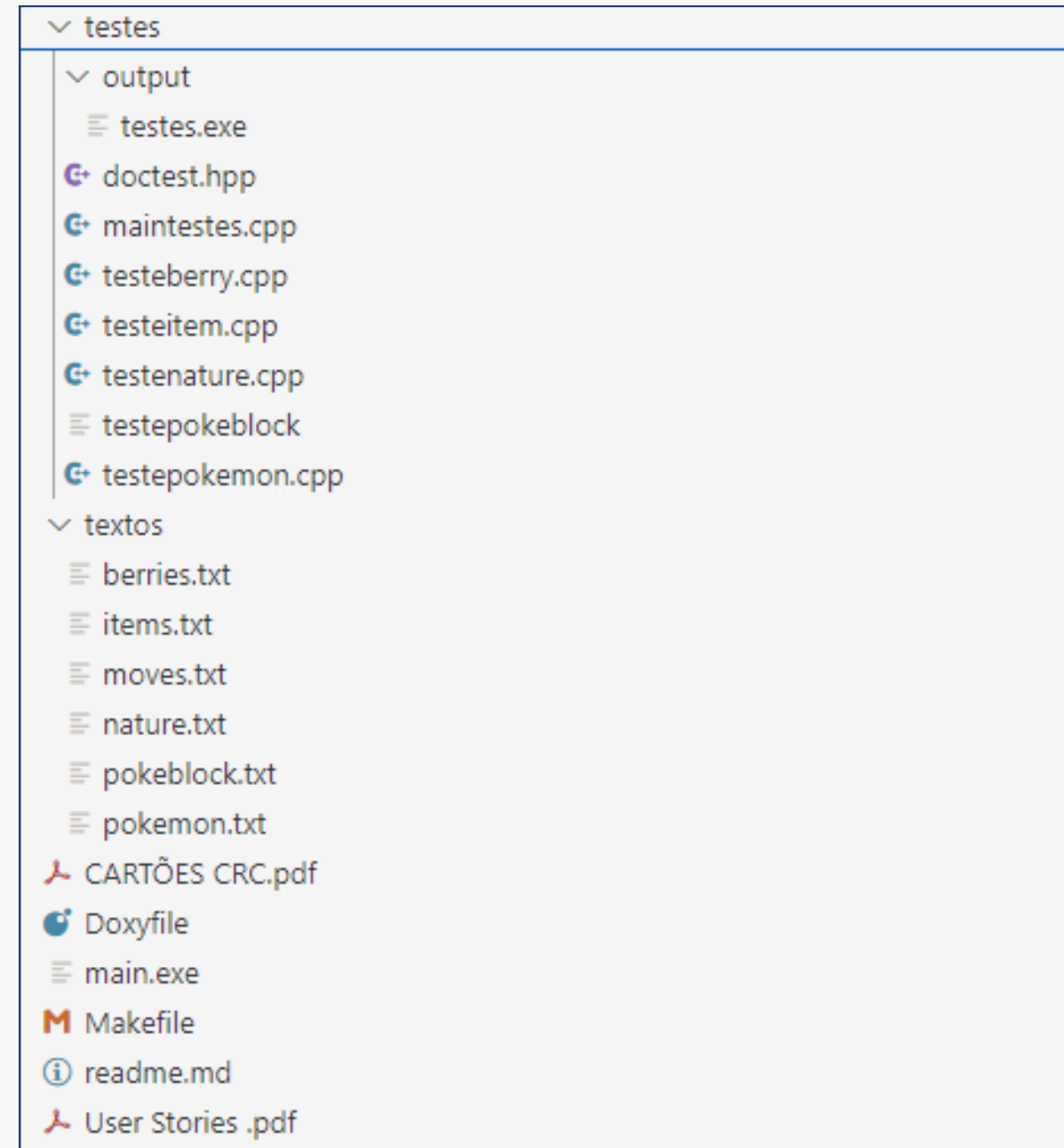
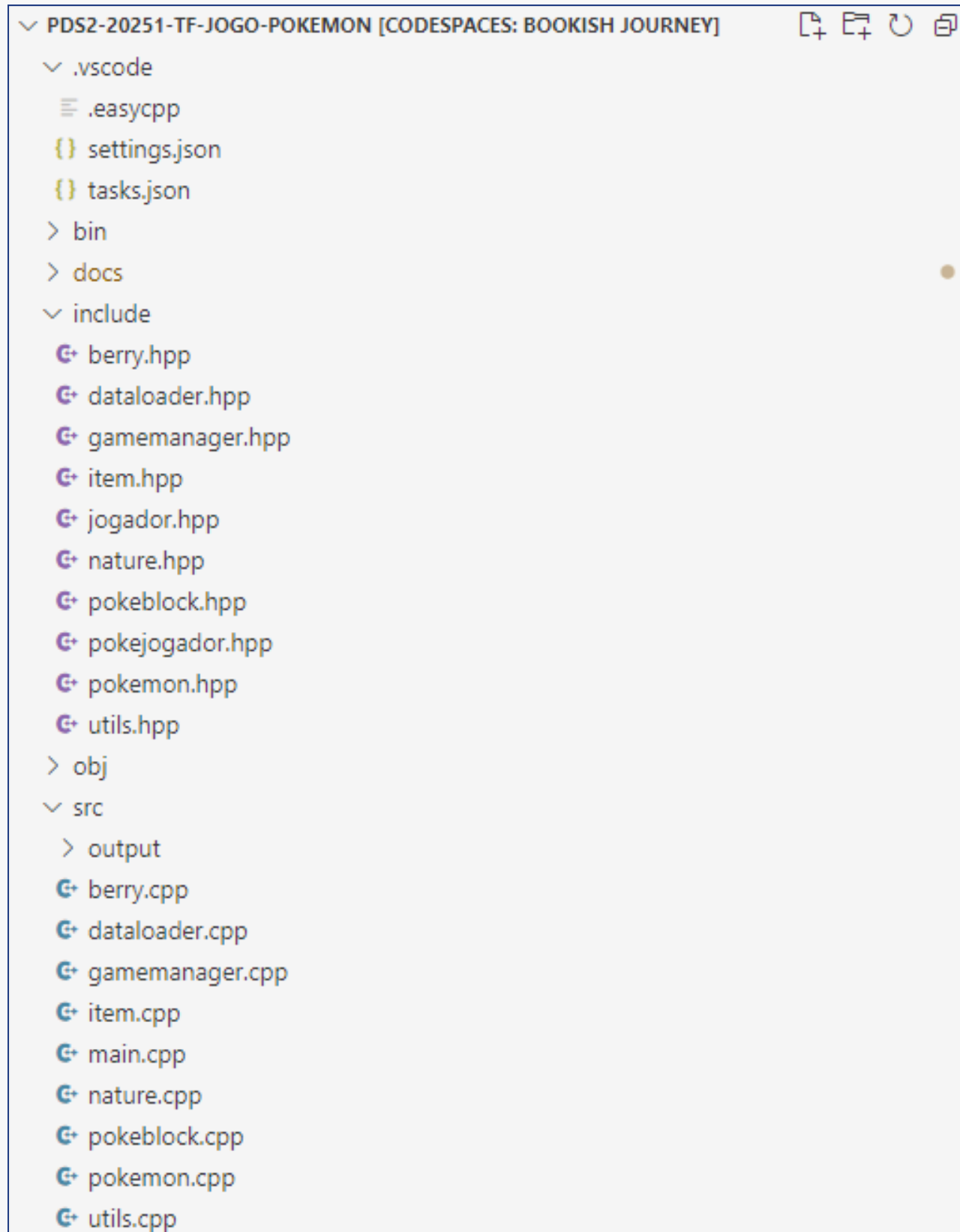
- Interface de texto estruturada com menus;
- Consulta avançada por filtros (ID, Nome, Tipo, Raridade);
- Dados carregados via CSV.



```
=== Bem-vindo ao Mundo Pokemon! ===  
Nenhum jogo salvo encontrado.  
Digite seu nome, treinador: Allan  
Seu Pokémon inicial é Bulbasaur!
```

```
=== Menu Principal ===  
1. Consultar Pokemons  
2. Consultar Itens  
3. Consultar Berries  
4. Consultar Pokeblocks  
5. Salvar Jogo  
6. Carregar Jogo  
0. Sair  
Escolha: █
```

Estrutura do Projeto



GameManager

A classe GameManager centraliza toda a lógica de fluxos do jogo:

- Carrega dados via DataLoader;
- Gerencia jogador e time;
- Gera IVs aleatórios e calcula stats;
- Menus interativos;
- Exibe dados formatados (Pokédex, Itens);
- Salva/Carrega progresso.

Carregamento modular com DataLoader

```
✓ void GameManager::carregarDados() {  
    berries = DataLoader::carregarBerries("textos/berries.txt");  
    items = DataLoader::carregarItens("textos/items.txt");  
    pokemons = DataLoader::carregarPokemons("textos/pokemon.txt");  
    pokeblocks = DataLoader::carregarPokeblocks("textos/pokeblock.txt");  
    natures = DataLoader::carregarNatures("textos/nature.txt");  
}
```

Não lê CSV diretamente

Usa DataLoader, garantindo a separação de responsabilidades

GameManager

Geração Aleatória de Atributos

```
int GameManager::gerarIV() {  
    return rand() % 32; // IV de 0 a 31
```

```
Nature GameManager::escolherNatureAleatoria() {  
    if(natures.empty()) throw std::runtime_error("Nenhuma natureza carregada");  
    int idx = rand() % natures.size();  
    return natures[idx];
```

Cada Pokémon gerado tem stats únicos.

Evita que todos os Pokémon sejam clones.

Dá mais realismo e jogabilidade, aproximando da mecânica dos jogos reais.

Potencial de Expansão

Elaboração em formato modular para facilitar possíveis expansões futuras.

- Métodos separados para cada tipo de consulta e menu;
- Dados carregados via DataLoader em métodos separados – se trocar o formato (CSV → banco de dados), muda só lá;
- Separação clara entre lógica de entrada/saída (menus) e lógica de dados.

DataLoader

A classe DataLoader centraliza toda a lógica de carregamento de dados do jogo:

- Cria os objetos a partir do arquivo texto
- Centraliza a leitura de arquivos
- Valida os dados de entrada

Carregamento modular com DataLoader

```
✓ void GameManager::carregarDados() {  
    berries = DataLoader::carregarBerries("textos/berries.txt");  
    items = DataLoader::carregarItens("textos/items.txt");  
    pokemons = DataLoader::carregarPokemons("textos/pokemon.txt");  
    pokeblocks = DataLoader::carregarPokeblocks("textos/pokeblock.txt");  
    natures = DataLoader::carregarNatures("textos/nature.txt");  
}
```


Testes

```
testes > testeberry.cpp
1  #include "doctest.hpp"
2  #include "../include/berry.hpp"
3  #include <stdexcept>
4  #include <string>
5
6  TEST_CASE("Entradas validas") {
7      std::string linha = "001,Cheri Berry,10,0,2,0,0,Common,Berry that slightly boosts Coolness";
8      Berry b = Berry::fromCSV(linha);
9
10     CHECK(b.getId() == 001);
11     CHECK(b.getNome() == "Cheri Berry");
12     CHECK(b.getPicante() == 10);
13     CHECK(b.getSeco() == 0);
14     CHECK(b.getDoce() == 2);
15     CHECK(b.getAmargo() == 0);
16     CHECK(b.getAzedo() == 0);
17     CHECK(b.getRaridade() == "Common");
18     CHECK(b.getDescricao() == "Berry that slightly boosts Coolness");
19 }
20
21 TEST_CASE("Campo Nome vazio") {
22     std::string linha = "001,,10,0,2,0,0,Common,Berry that slightly boosts Coolness";
23     CHECK_THROWS_AS(Berry::fromCSV(linha), std::invalid_argument);
24 }
25
26 TEST_CASE("Campo raridade vazio") {
27     std::string linha = "001,Cheri Berry,10,0,2,0,0,,Berry that slightly boosts Coolness";
28     CHECK_THROWS_AS(Berry::fromCSV(linha), std::invalid_argument);
29 }
30
31 TEST_CASE("Campo Descricao vazio") {
32     std::string linha = "001,Cheri Berry,10,0,2,0,0,Common,";
33     CHECK_THROWS_AS(Berry::fromCSV(linha), std::invalid_argument);
34 }
```

Conclusão

Aprendizados e Aplicação de Programação orientada a objetos;

- Encapsulamento com classes bem definidas;
- Modularização e separação de responsabilidades;
- Centralização de lógica com GameManager;
- Tratamento básico de exceções;
- Testes automatizados.

Resultados:

- Consolidação do conteúdo ministrado na disciplina;
 - Projeto funcional e organizado;
 - Código preparado para evoluções futuras.
- 

Obrigado!

